

A. Appendix

A.1. Background

A.1.1. ROBUSTNESS FOR LLMs

In the context of LLM safety research, an *aligned model* can be defined as one whose generations follow the rules and guidelines laid out by its developer. Such an LLM may be adversarially attacked via methods that produce inputs that cause the model to deviate from the desired guidelines (also called *jailbreaks*). We consider a model more *robust* or more *aligned* if it is more “difficult” to elicit the undesired behavior.

In contrast to computer vision, where standardized threat models like norm constraints can precisely capture the “difficulty” of breaking a model (Goodfellow et al., 2015; Madry et al., 2018), there is currently no consensus on how to best define difficulty in the context of LLM robustness. For practical purposes, models are typically ranked according to their robustness against a suite of existing attack algorithms (Zou et al., 2023b). Models that are broken less often or require more iterations to break are considered more robust. Most research, including this paper, is not focused on “true” worst-case robustness, which demands that models never produce undesired outputs given *any* input, and instead focus on real-world scenarios and algorithms to find practically usable jailbreaks in reasonable time (Zou et al., 2024). The terms are also used somewhat inconsistently, with “robustness” often referring more to the stability of a model’s output with respect to constrained adversarial perturbations of its input rather than the ability to follow a developer’s intentions.

More formally, we investigate a model f_θ , which maps from the space of input token sequences $\mathcal{X}$ to output token sequences $\mathcal{Y}$. We also define a set of *admissible* input token sequences $\mathcal{X}^* \subset \mathcal{X}$ which consists of all token sequences that are reachable by tokenizing a user-submitted string (i.e., they do not contain control tokens or impossible-to-reach sequences due to token merging). We consider attacks that find an admissible sequence of discrete input tokens $\hat{x} \in \mathcal{X}^*$ that causes its response $y \sim f_\theta(\hat{x})$ to conflict with the guidelines set out by the developer. Deciding whether a generation conflicts with the developer’s guidelines is likely an ill-defined problem due to the ambiguity of language. In practice, the community uses judging functions J which map from model generations $y \in \mathcal{Y}$ to either discrete success flags $\{0, 1\}$ or continuous scores $[0, 1]$. J is instantiated either via human evaluation or, more commonly, as a finetuned LLM trained on data of human evaluations of harmful and harmless generations.

A.1.2. ROBUSTNESS IN THE IMAGE DOMAIN

In the past, the computer vision robustness research community has struggled with reproducibility issues and unreliable defenses which were often quickly broken after their release (Athalye et al., 2018; Carlini, 2024; Uesato et al., 2018).

Clearly defining commonly used threat models, such as $L_\infty = 8/255$ norm constraints, which provide a precise measure of the “difficulty” of breaking a model (Goodfellow et al., 2015; Madry et al., 2018), have made results in field more reliable by making attacks and defense easily comparable and testable. Additionally, public leaderboards with consistent datasets and threat models have been used to track progress (Croce et al., 2020). The community also focused on more standardized frameworks (Papernot et al., 2016; Croce & Hein, 2020; Kim, 2020) and datasets (Krizhevsky et al., 2009), making results easier to reproduce and avoiding noise resulting from slightly different implementations.

A.2. Case Study: Effects of White-space Tokens on GCG Against Llama 2

We run GCG (Zou et al., 2023b) against `Llama-2-7b-chat-hf` using default hyperparameters. We use the `Meta chat` template without system prompt (as recommended), and report the mean across the first 100 prompts from HarmBench (Mazeika et al., 2024). In one trial, we do not account for the SentencePiece (Kudo, 2018) underline token (ID 29781), which Llama 2 predicts at the beginning of each of its responses, before continuing with regular text-tokens. This token is not added automatically by the tokenizer, even with the correct template, and must be manually inserted to ensure correctness.

We see that GCG makes faster progress when the white-space token is correctly handled, achieving a 91% many-trial ASR after 250 iterations, whereas the misaligned implementation only reaches 63%. The loss curve further illustrates this effect, with the correct implementation rapidly reaching more than 50% lower loss values.

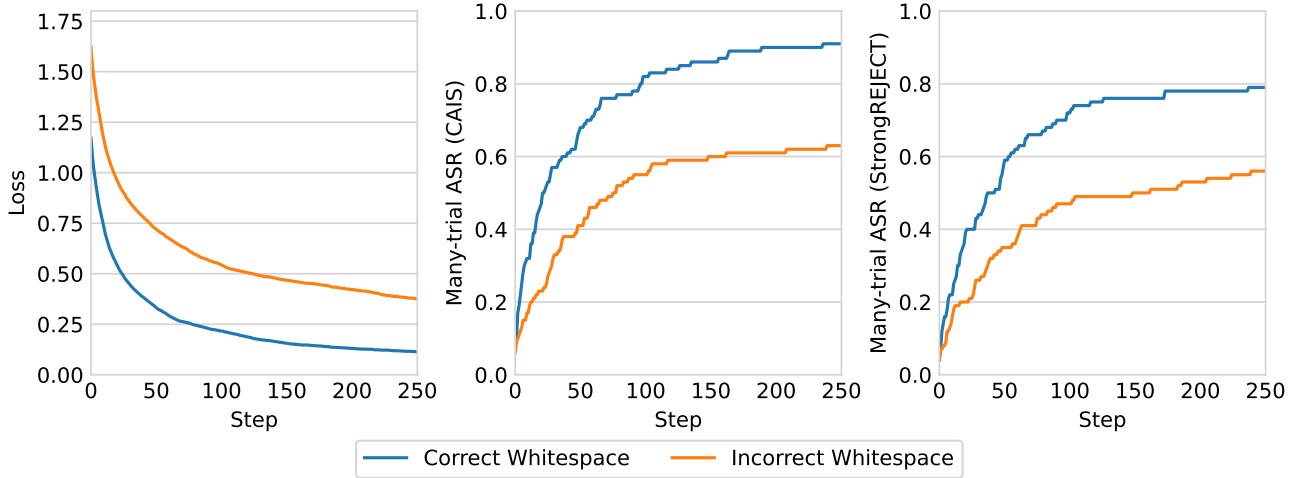


Figure 4: Impact of white-space tokens on GCG attack progress vs. Llama-2-7b-chat-hf.

A.3. Case Study: Effects of Implementation Details on Optimization

We use the first 100 prompts from HarmBench (Mazeika et al., 2024) and run GCG (Zou et al., 2023b) against Llama-3.1-8B-Instruct. We use the default hyperparameters for GCG (batch size 512, number of steps 250) and use the BF16 floating point format for the model weights. Our reference implementation of GCG is based on the official nanoGCG repository but includes several enhancements.

- **Chat Template:** Instead of the default chat template provided by HuggingFace, we use the Meta-recommended template.
- **Token Filter for Non-Admissible Inputs:** While nanoGCG includes a filter to remove special tokens (control/role/system) tokens from the optimization, we design a stricter token filter which fixes a few issues. To evaluate whether an attack prompt is admissible, nanoGCG decodes and re-encodes the attack tokens and verifies that they remain unchanged. While this approach is sufficient for most scenarios, in some cases, the tokenizer merges characters *across the boundary* of the attack suffix and the remaining prompt, which is not detected by the filter. This can lead to inadmissible token sequences which cannot be reached from standard input strings and are therefore filtered by our implementation.
- **Filtering of Control Tokens:** The reference implementation fails to filter control tokens which were added to the tokenizer post-hoc, but can affect the optimization and should always be excluded.

In Figure 5, we demonstrate that these subtle changes can have a significant effect on the performance of GCG, causing an ASR difference of up to 14% among configurations which use a system prompt. We also include results without the system prompt to demonstrate the importance of following the instructions by model developers to include system prompts where required. The main point of this experiment is not to optimize for more efficient algorithms or higher ASR, but simply to show that minor details can significantly affect the outcomes of LLM attack evaluations.

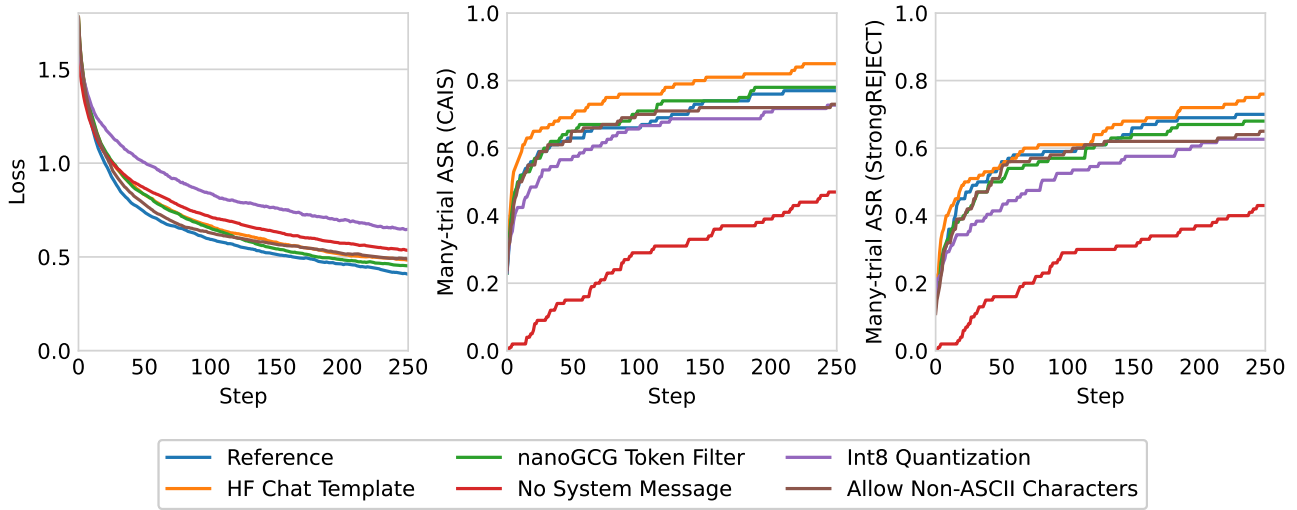


Figure 5: Impact of implementation details on GCG attack progress vs. Llama-3.1-8B-Instruct.